

1. cv2.imread()

Reads an image from a file.

Attribute	Meaning	Default	Options
filename	Path to the image file	Required	Any valid file path
flags	Specifies how the image should be read	cv2.IMREAD_COLOR	cv2.IMREAD_COLOR (1) – Loads color image, ignores alpha cv2.IMREAD_GRAYSCALE (0) – Loads in grayscale mode cv2.IMREAD_UNCHANGED (-1) – Loads image as-is including alpha channel

Exam tip: You can pass integers directly: 1, 0, or -1.

2. cv2.cvtColor()

Converts an image from one color space to another.

Attribute	Meaning	Default	Options
src	Input image	Required	Any valid image array

code	Color conversion code	Required	cv2.COLOR_BGR2GRAY (6) cv2.COLOR_BGR2RGB (4) cv2.COLOR_RGB2BGR (4) cv2.COLOR_BGR2BGR A (0) cv2.COLOR_BGRA2BGR (1) cv2.COLOR_BGR2XYZ (32) cv2.COLOR_BGR2YCrCb (36) cv2.COLOR_GRAY2BGR (8) cv2.COLOR_GRAY2RGB (8) ...and many more
------	-----------------------	----------	---

Common conversions in the lab:

- cv2.COLOR_BGR2RGB – Convert BGR (OpenCV default) to RGB (for Matplotlib)
- cv2.COLOR_BGR2GRAY – Convert to grayscale for edge detection

3. cv2.Sobel()

Calculates the first, second, or mixed image derivatives using an extended Sobel operator.

Attribute	Meaning	Default	Options
src	Input image	Required	Grayscale image recommended
ddepth	Output image depth	Required	-1 (same as input) cv2.CV_16S (16-bit signed) cv2.CV_32F (32-bit float) cv2.CV_64F (64-bit float) – used in lab to

			preserve negative values
dx	Order of derivative in x direction	Required	0 or 1 (typically 1 for x-gradient)
dy	Order of derivative in y direction	Required	0 or 1 (typically 1 for y-gradient)
ksize	Size of the extended Sobel kernel	3	1, 3, 5, 7
scale	Optional scale factor for computed derivative values	1	Any positive float
delta	Optional delta added to results before storing	0	Any float
borderType	Pixel extrapolation method	cv2.BORDER_DEFAULT	cv2.BORDER_CONSTANT cv2.BORDER_REPLICATE cv2.BORDER_REFLECT cv2.BORDER_WRAP cv2.BORDER_REFLECT_101

Exam tip: Use cv2.CV_64F to keep negative gradients. ksize must be odd and between 1 and 7.

4. cv2.GaussianBlur()

Blurs an image using a Gaussian filter.

Attribute	Meaning	Default	Options
src	Input image	Required	Any valid image

ksize	Gaussian kernel size	Required	(width, height) – both must be positive odd numbers, or (0,0) to compute from sigma
sigmaX	Gaussian kernel standard deviation in X direction	Required	Any positive float
sigmaY	Gaussian kernel standard deviation in Y direction	0	0 – set equal to sigmaX; if both are 0, computed from ksize
borderType	Pixel extrapolation method	cv2.BORDER_DEFAULT	cv2.BORDER_CONSTANT cv2.BORDER_REPLICATE cv2.BORDER_REFLECT cv2.BORDER_REFLECT_101 cv2.BORDER_WRAP (not supported)
Exam tip: ksize=(5,5) and sigmaX=1.4 are used in the lab for Canny and LoG preprocessing.			

5. cv2.Canny()

Finds edges in an image using the Canny algorithm.

Attribute	Meaning	Default	Options
image	8-bit single-channel input image	Required	Grayscale image
threshold1	First threshold for hysteresis (lower threshold)	Required	Any double (e.g., 50)

threshold2	Second threshold for hysteresis (upper threshold)	Required	Any double (e.g., 150)
apertureSize	Aperture size for the Sobel operator	3	3, 5, 7
L2gradient	Flag for more accurate gradient magnitude	False	True – uses L2 norm: $\sqrt{Gx^2 + Gy^2}$ False – uses L1 norm: $ Gx + Gy $

Exam tip: The smaller threshold is used for edge linking; the larger for finding initial strong edges.

6. cv2.Laplacian()

Calculates the Laplacian of an image.

Attribute	Meaning	Default	Options
src	Input image	Required	Grayscale recommended
ddepth	Desired depth of destination image	Required	-1 (same as input) cv2.CV_16S cv2.CV_32F cv2.CV_64F – used in lab
ksize	Aperture size for second-derivative filters	1	Must be positive and odd: 1, 3, 5, 7
scale	Optional scale factor	1	Any positive float
delta	Optional delta added to results	0	Any float

borderType	Pixel extrapolation method	cv2.BORDER_DEFAULT	cv2.BORDER_CONSTANT cv2.BORDER_REPLICATE cv2.BORDER_REFLECT cv2.BORDER_REFLECT_101
------------	----------------------------	--------------------	---

Exam tip: When ksize=1, a 3×3 kernel is used: $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$.

7. cv2.convertScaleAbs()

Converts the result to absolute values and scales to 8-bit unsigned.

Attribute	Meaning	Default	Options
src	Input array	Required	Any valid image/array
alpha	Scale factor (contrast control)	1	Any float (e.g., 1.0 to 3.0)
beta	Delta added after scaling (brightness control)	0	Any float (e.g., 0 to 100)

Exam tip: Used after Laplacian to visualize the result properly.

8. cv2.HoughLines()

Detects lines in a binary image using the standard Hough Transform.

Attribute	Meaning	Default	Options
image	8-bit single-channel binary source image	Required	Edge map (e.g., from Canny)
lines	Output vector of lines	Required	Each line is (p, θ)

rho	Distance resolution of accumulator in pixels	Required	Any float (e.g., 1)
theta	Angle resolution of accumulator in radians	Required	Any float (e.g., $\text{np.pi}/180$)
threshold	Accumulator threshold parameter	Required	Any int – only lines with enough votes returned
srn	Divisor for distance resolution (multi-scale)	0	0 for classic Hough; positive for multi-scale
stn	Divisor for angle resolution (multi-scale)	0	0 for classic Hough; positive for multi-scale
min_theta	Minimum angle to check for lines	0	Must be between 0 and max_theta
max_theta	Maximum angle to check for lines	π (CV_PI)	Must be between min_theta and π

Exam tip: Line represented as $\rho = x \cdot \cos\theta + y \cdot \sin\theta$ in parameter space.

9. cv2.HoughCircles()

Detects circles in a grayscale image using the Hough Gradient method.

Attribute	Meaning	Default	Options
image	8-bit single-channel grayscale input image	Required	Grayscale image
circles	Output vector of found circles	Required	Each circle is (x, y, radius)

method	Detection method	Required	cv2.HOUGH_GRADIENT – only implemented method
dp	Inverse ratio of accumulator resolution to image resolution	Required	1 – same resolution 2 – accumulator halved
minDist	Minimum distance between centers of detected circles	Required	Any float – too small → multiple false circles; too large → misses circles
param1	First method-specific parameter	100	Higher threshold for Canny edge detector (lower = half of this)
param2	Second method-specific parameter	100	Accumulator threshold for circle centers – lower → more false circles
minRadius	Minimum circle radius	0	Any int
maxRadius	Maximum circle radius	0	Any int
Exam tip: Circle equation: $(x-a)^2 + (y-b)^2 = r^2$. Parameter space is 3D: (a, b, r).			

10. cv2.SIFT_create()

Creates a SIFT detector object.

Attribute	Meaning	Default	Options
nfeatures	Number of best features to retain	0 (no limit)	Any positive int

nOctaveLayers	Number of layers in each octave	3	Any positive int
contrastThreshold	Contrast threshold to filter weak features	0.04	Any positive float – larger → fewer features
edgeThreshold	Threshold to filter edge-like features	10	Any positive float – larger → more features retained
sigma	Gaussian sigma applied at octave #0	1.6	Any positive float – reduce for soft-lens images
descriptorType	Descriptor type	CV_32F	CV_32F
enable_precise_upscale	Precise upscale flag	False	True / False

Exam tip: contrastThreshold is divided by nOctaveLayers during filtering. For the value in Lowe's paper (0.03) with default nOctaveLayers, set this to 0.09.

11. sift.detectAndCompute()

Detects keypoints and computes descriptors.

Attribute	Meaning	Default	Options
image	Input grayscale image	Required	Grayscale image
mask	Mask specifying where to look for keypoints	None	None or a binary mask

Returns:

- keypoints – List of KeyPoint objects
- descriptors – NumPy array of shape (N, 128) where N = number of keypoints

Exam tip: Descriptor size is always 128 for SIFT (4×4×8).

12. cv2.drawKeypoints()

Draws keypoints on an image.

Attribute	Meaning	Default	Options
image	Source image	Required	Any valid image
keypoints	Keypoints to draw	Required	List of KeyPoint objects
outImage	Output image	Required	Any valid image array
color	Color of keypoints	Scalar::all(-1) (random)	Any BGR scalar, e.g., (255, 0, 0)
flags	Drawing flags	cv2.DRAW_MATCHES_FLAGS_DEFAULT	cv2.DRAW_MATCHES_FLAGS_DEFAULT – draws small circles cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS – draws circles with size and orientation cv2.DRAW_MATCHES_FLAGS_DRAW_OVER_OUTIMG – draws on existing output cv2.DRAW_MATCHES_FLAGS_NOT_DRAW_SINGLE_POINTS – skips single keypoints

Exam tip: Use DRAW_RICH_KEYPOINTS to visualize keypoint size (scale) and orientation.

13. cv2.GaussianBlur() – Already Covered Above (see #4)

14. np.sqrt() – NumPy Function

Calculates the square root, used for gradient magnitude: $G = \sqrt{G_x^2 + G_y^2}$.

Attribute	Meaning	Default	Options
x	Input array	Required	Any NumPy array

Quick Exam Revision Table

Function	Key Parameters to Remember
imread	flags: IMREAD_COLOR (1), IMREAD_GRAYSCALE (0), IMREAD_UNCHANGED (-1)
cvtColor	code: COLOR_BGR2GRAY (6), COLOR_BGR2RGB (4)
Sobel	ddepth: CV_64F; dx, dy: 0 or 1; ksize: 1, 3, 5, 7
GaussianBlur	ksize: odd positive; sigmaX, sigmaY: 0 or positive
Canny	threshold1 (lower), threshold2 (upper); apertureSize: 3, 5, 7; L2gradient: True/False
Laplacian	ddepth: CV_64F; ksize: 1, 3, 5, 7
convertScaleAbs	alpha (contrast), beta (brightness)
HoughLines	rho, theta, threshold; min_theta, max_theta
HoughCircles	method: HOUGH_GRADIENT; dp, minDist, param1, param2, minRadius, maxRadius
SIFT_create	nfeatures, nOctaveLayers: 3, contrastThreshold: 0.04, edgeThreshold: 10, sigma: 1.6

drawKeypoints

flags:

DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS

Good luck on your exam on Thursday! Focus on the parameter names, their default values, and the options available for each.